

Sri Lanka Institute of Information Technology



IT3012: Intelligent Agents - Lab 4

Y3.S1.WD.AI.01.02

Malabe Campus

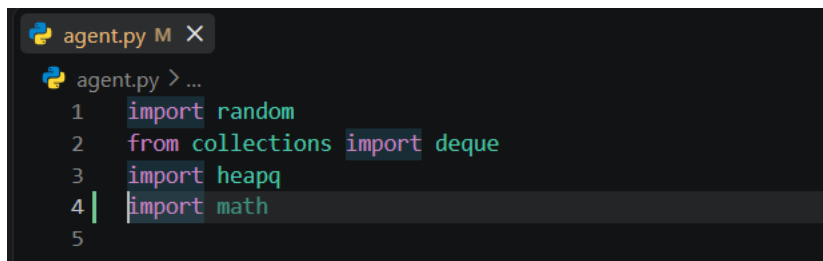
Student ID -	IT24103099
Student Name -	Uthpala P.B.H
Module Name -	Intelligent Agents
Module Number -	IT3012
LabSheet Number -	4
LabSheet Date -	2026/August/23

Practical Implementation - Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

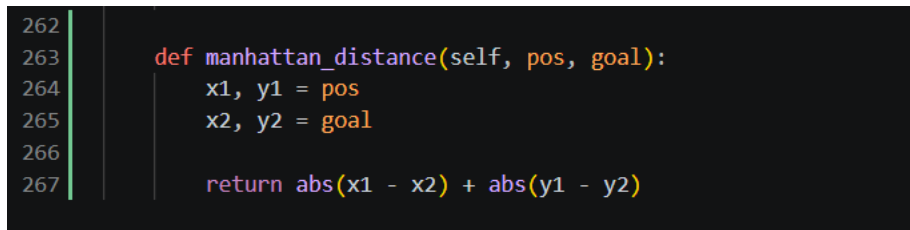
Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called Week_04 in your Forked Github Repo.
2. Open agent.py and locate your SearchAgent class[cite: 1].



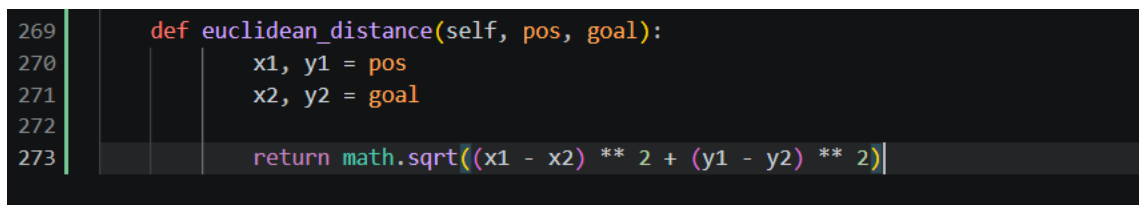
```
agent.py M X
agent.py > ...
1 import random
2 from collections import deque
3 import heapq
4 import math
5
```

3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.



```
262
263     def manhattan_distance(self, pos, goal):
264         x1, y1 = pos
265         x2, y2 = goal
266
267         return abs(x1 - x2) + abs(y1 - y2)
```

4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to import math for this.



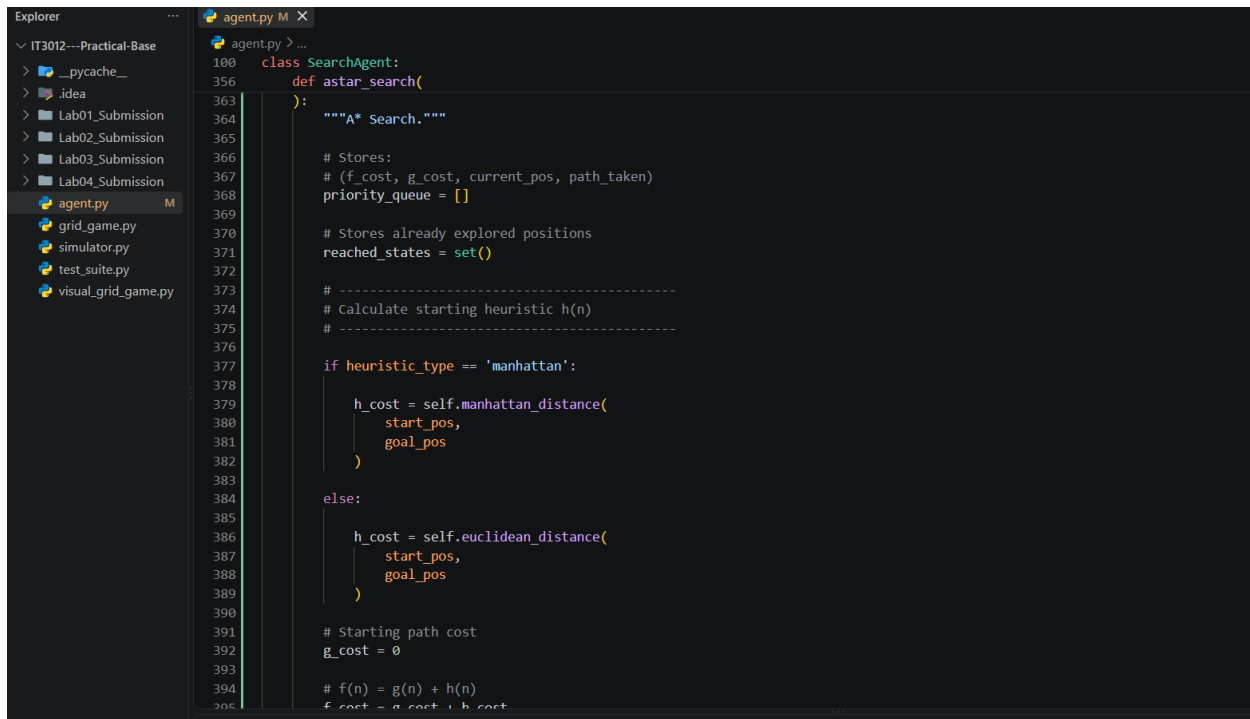
```
269     def euclidean_distance(self, pos, goal):
270         x1, y1 = pos
271         x2, y2 = goal
272
273         return math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2)
```

5. Testing Checkpoint: Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside SearchAgent , create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')` .
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states` .
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)` . For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard while loop to process the queue. Pop the node with the lowest `f_cost` . If `current_pos == goal_pos` , return the `path_taken` . Otherwise, add `current_pos` to `reached_states` .
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\text{new}} = g_{\text{current}} + 1$, h_{new} using your heuristic, and $f_{\text{new}} = g_{\text{new}} + h_{\text{new}}$. Push the new tuple to the priority queue



```
100 class SearchAgent:
356 def astar_search(
363 ):
364     """A* Search."""
365
366     # Stores:
367     # (f_cost, g_cost, current_pos, path_taken)
368     priority_queue = []
369
370     # Stores already explored positions
371     reached_states = set()
372
373     # -----
374     # Calculate starting heuristic h(n)
375     # -----
376
377     if heuristic_type == 'manhattan':
378         h_cost = self.manhattan_distance(
379             start_pos,
380             goal_pos
381         )
382
383     else:
384         h_cost = self.euclidean_distance(
385             start_pos,
386             goal_pos
387         )
388
389     # Starting path cost
390     g_cost = 0
391
392     # f(n) = g(n) + h(n)
393     f_cost = g_cost + h_cost
```

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent` .
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing `BFS/DFS/UCS`.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos` . Call your `astar_search` method and store the returned list of actions in `self.plan` .
5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()` . Run the simulation and observe the optimized pathfinding!

```
# =====
# LAB 04 STEP 1.3
# AGENT DECISION LOOP
# =====

def sense_and_act(self, percept: dict) -> str:
    """
    Create a complete plan and execute
    it one action at a time.
    """

    # Create a new plan only when
    # the current plan is empty
    if not self.plan:

        # Current position of the agent
        start = tuple(
            | percept['agent_pos']
        )

        # Remaining food positions
        all_food = percept['all_food']

        # Wall positions
        walls = set(
            | percept['walls']
        )

        # Grid dimensions
        grid_size = percept['grid_size']

        # No food remains
```

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

- Uniform-Cost Search (UCS) prioritizes the node with the **lowest path cost $g(n)$** from the starting point. A* Search prioritizes the node with the **lowest estimated total cost $f(n) = g(n) + h(n)$** , where $h(n)$ estimates the remaining distance to the goal. Therefore, A* uses heuristic information to guide the search toward the goal.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

- Manhattan Distance is admissible for a 4-way movement grid because the agent can only move **Up, Down, Left, or Right**. The Manhattan Distance gives the minimum number of moves required to reach the goal when there are no obstacles, so it never overestimates the actual shortest path cost.
- If the heuristic were not admissible and overestimated the true cost, A* could choose the wrong nodes and **would no longer be guaranteed to find the optimal shortest path**.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

- No. If diagonal movement is allowed at an appropriate diagonal cost, Manhattan Distance can overestimate the actual shortest path because the agent can reduce both the horizontal and vertical distance in one diagonal move. Therefore, it would no longer be an admissible heuristic in that setting.
- For 8-way movement, we should switch to **Euclidean Distance** when diagonal movement uses geometric costs such as $\sqrt{2}$. If all eight moves have the same cost, **Chebyshev Distance** is the more appropriate heuristic.

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

- A stronger heuristic could consider not only the distance to the closest food but also the cost required to connect and visit **all remaining food items**.

For example, we can calculate the distance from the agent to the nearest food and add the cost of a **Minimum Spanning Tree (MST)** connecting all remaining food positions. This provides a better estimate of the total remaining cost because it considers the distribution of all food items rather than only the closest one.

A simple expression is:

$h(n) = \text{distance to nearest food} + \text{MST cost of all remaining food}$

This would guide A* more effectively when the goal is to collect all remaining food.